



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

Trabalho Prático - Fase 2

Grupo 31

Afonso Martins
a106931

Gonçalo Castro
a107337

Francisco Lage
a106813

Ricardo Neves
a106850

Março, 2026

CG

Resumo

Este documento descreve o desenvolvimento da 2ª fase do projeto prático da Unidade Curricular de Computação Gráfica, lecionada no 2.º semestre do 3.º ano da Licenciatura em Engenharia Informática.

O objetivo desta fase consistiu em estender o motor da fase anterior com suporte a transformações geométricas hierárquicas — translação, rotação e escala — aplicadas a grupos de modelos organizados em árvore. Como cena de demonstração obrigatória, foi construída uma versão estática do sistema solar, com o sol, planetas e respectivas luas posicionados através desta hierarquia. Foi ainda desenvolvido, como funcionalidade extra, um inspetor de cena integrado na interface ImGui, que permite visualizar a hierarquia completa da cena em tempo real.

Índice

1. Leitura do XML	1
1.1. Processo de Leitura	1
1.2. Ordem das Transformações	2
1.3. Compatibilidade com Formato Legado	2
2. Transformações Geométricas	2
2.1. Translação	2
2.2. Rotação	3
2.3. Escala	3
2.4. Ordem de Aplicação	3
3. Sistema Solar	3
3.1. Decisões de Design	3
3.2. Hierarquia da Cena	4
4. Inspetor de Cena	5
4.1. Implementação	5
4.2. Utilidade	5
5. Testes	6
6. Conclusão	7

Lista de Figuras

Figura 1 Sistema Solar	4
Figura 2 Inspetor de Cena no painel ImGui	6
Figura 3 Teste 1 — Cubo com translate, rotate e scale	6
Figura 4 Teste 2 — Hierarquia: caixa, cone e esfera	6
Figura 5 Teste 3 — Boneco de neve	7
Figura 6 Teste 4 — Caixas em círculo	7

1. Leitura do XML

A leitura do ficheiro XML foi estendida para suportar o novo elemento `<transform>`, que pode conter qualquer combinação de `<translate>`, `<rotate>` e `<scale>`. A função responsável pela construção da hierarquia em memória é `initializeGroupFromXML()`, que opera de forma recursiva sobre a árvore XML.

1.1. Processo de Leitura

O elemento `<group>` é a unidade estrutural fundamental da cena. Cada grupo agrega um conjunto de transformações, um conjunto de modelos e uma lista de grupos filhos. Esta organização permite construir hierarquias arbitrariamente profundas, onde cada nível herda o contexto de transformação do nível acima.

Para cada `<group>` encontrado, a função começa por iterar sobre todos os elementos `<transform>` que este contém. Um `<transform>` funciona como um contentor de operações geométricas — dentro dele podem existir um ou mais `<translate>`, `<rotate>` e `<scale>`, pela ordem que o utilizador definir. Suportar múltiplos blocos `<transform>` por grupo foi uma decisão deliberada: permite separar logicamente diferentes tipos de operações geométricas dentro do mesmo grupo, tornando os ficheiros de cena mais organizados e fáceis de editar sem alterar o comportamento da cena.

Após as transformações, são carregados os modelos referenciados no bloco `<models>`. Para cada `<model>`, o motor tenta localizar o ficheiro em vários caminhos pré-definidos. Se o ficheiro não for encontrado, é emitido um aviso na consola e o modelo é ignorado, sem interromper o carregamento da restante cena.

Por fim, a função invoca-se a si própria recursivamente para cada `<group>` filho, construindo assim a árvore completa em memória antes de qualquer renderização.

O formato XML suportado é o seguinte:

```
<group>
  <transform>
    <translate x="4" y="0" z="0" />
    <rotate angle="30" x="0" y="1" z="0" />
  </transform>
  <transform>
    <scale x="2" y="0.3" z="1" />
  </transform>
  <models>
    <model file="assets/models/cone.3d" />
  </models>
  <group>
    <!-- grupo filho – herda as transformações acumuladas do pai -->
  </group>
</group>
```

1.2. Ordem das Transformações

A ordem de leitura é crítica, uma vez que as transformações geométricas não são comutativas. Aplicar primeiro uma rotação e depois uma translação produz um resultado completamente diferente da ordem inversa, como ilustrado abaixo:

```
Rotate(90°, Y) → Translate(5, 0, 0)
    objeto roda no lugar e depois move-se ao longo do novo eixo X
    resultado: objeto em (0, 0, -5)

Translate(5, 0, 0) → Rotate(90°, Y)
    objeto move-se e depois roda em torno da origem
    resultado: objeto em (0, 0, 5) ← posição diferente
```

Listagem 1: Exemplo da não-comutatividade das transformações geométricas. A ordem de declaração no XML determina o resultado final.

A implementação respeita estritamente esta ordem, garantindo que o comportamento da cena corresponde exactamente ao que o utilizador declarou no ficheiro XML.

1.3. Compatibilidade com Formato Legado

O motor mantém compatibilidade com o formato da fase anterior, que usava uma lista plana de `<models>` directamente em `<world>` sem grupos. Se o XML contiver um elemento `<group>` de raiz, este é usado para construir a hierarquia; caso contrário, os modelos são adicionados directamente ao grupo raiz sem transformações.

2. Transformações Geométricas

As transformações são aplicadas em `Group::render()` recorrendo à pilha de matrizes do OpenGL. A cada grupo, é chamado `glPushMatrix()` antes de aplicar as suas transformações e `glPopMatrix()` no final. Este mecanismo isola o estado de cada grupo — as suas transformações afetam apenas os seus modelos e filhos, nunca os grupos irmãos ou o pai.

Os grupos filhos herdam automaticamente a matriz acumulada do pai. Isto significa que a posição de um filho é sempre relativa ao pai, o que é o comportamento natural de um grafo de cena hierárquico e é fundamental para construir cenas como o sistema solar, onde as luas devem orbitar os planetas independentemente da posição destes, que constitui a demonstração esperada para esta fase do trabalho.

2.1. Translação

A translação desloca um grupo no espaço segundo um vetor (x, y, z) , aplicada via `glTranslatef(x, y, z)`. É a transformação mais usada na cena do sistema solar para posicionar planetas e luas, no exemplo que mostraremos mais adiante. A matriz correspondente é:

$$T(x, y, z) = \begin{pmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & y \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2.2. Rotação

A rotação roda um grupo em torno de um eixo arbitrário (x, y, z) por um ângulo a em graus, aplicada via `glRotatef(a, x, y, z)`. O uso de um eixo arbitrário, em vez de eixos principais fixos, confere maior flexibilidade — permite, por exemplo, inclinar planetas em relação ao plano orbital com um único parâmetro (exemplo que mostraremos numa secção a seguir). A matriz correspondente é:

$$R(x, y, z, a) = \begin{pmatrix} x^2 + (1 - x^2) \cos a & xy(1 - \cos a) - z \sin a & xz(1 - \cos a) + y \sin a & 0 \\ xy(1 - \cos a) + z \sin a & y^2 + (1 - y^2) \cos a & yz(1 - \cos a) - x \sin a & 0 \\ xz(1 - \cos a) - y \sin a & yz(1 - \cos a) + x \sin a & z^2 + (1 - z^2) \cos a & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2.3. Escala

A escala redimensiona um grupo de forma independente em cada eixo, aplicada via `glScalef(x, y, z)`. Uma escala uniforme ($x = y = z$) preserva as proporções do modelo; uma escala não uniforme permite achatamentos e alongamentos. A matriz correspondente é:

$$S(x, y, z) = \begin{pmatrix} x & 0 & 0 & 0 \\ 0 & y & 0 & 0 \\ 0 & 0 & z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2.4. Ordem de Aplicação

Dentro de cada grupo, as transformações são aplicadas sequencialmente pela ordem em que foram lidas do XML. O OpenGL acumula estas operações multiplicando cada nova matriz pela matriz corrente da pilha. O resultado final de um grupo com translação T , rotação R e escala S é:

$$M_{\text{final}} = M_{\text{pai}} \times T \times R \times S$$

onde M_{pai} é a matriz herdada do grupo pai.

3. Sistema Solar

Foi construída uma cena estática do sistema solar com o sol, seis planetas e respectivas luas, organizados numa hierarquia de grupos.

3.1. Decisões de Design

A escala dos corpos celestes foi ajustada para tornar a cena visualmente informativa sem recorrer a valores reais — as distâncias reais entre planetas tornariam a cena impossível de visualizar de um único ponto de câmara. Foram tomadas as seguintes decisões:

- O sol ocupa a origem sem transformações, servindo de âncora visual para toda a cena.
- Os planetas estão distribuídos ao longo do eixo X , com espaçamento crescente para evitar sobreposição à medida que os planetas ficam maiores.
- As escalas dos planetas são proporcionais ao seu tamanho relativo — o planeta 5 (análogo a Júpiter) tem escala 1.5, enquanto os planetas interiores têm escalas menores.

- As luas estão posicionadas em coordenadas locais do planeta pai, pelo que a sua translação é relativa ao planeta e não à origem — a herança de transformações trata disto automaticamente.
- Alguns planetas incluem elementos decorativos gerados com primitivas da fase anterior. O planeta 6, por exemplo, inclui um anel gerado com o torus, escalado de forma não uniforme ($x = 2, y = 0.2, z = 2$) para simular um anel planetário achatado.

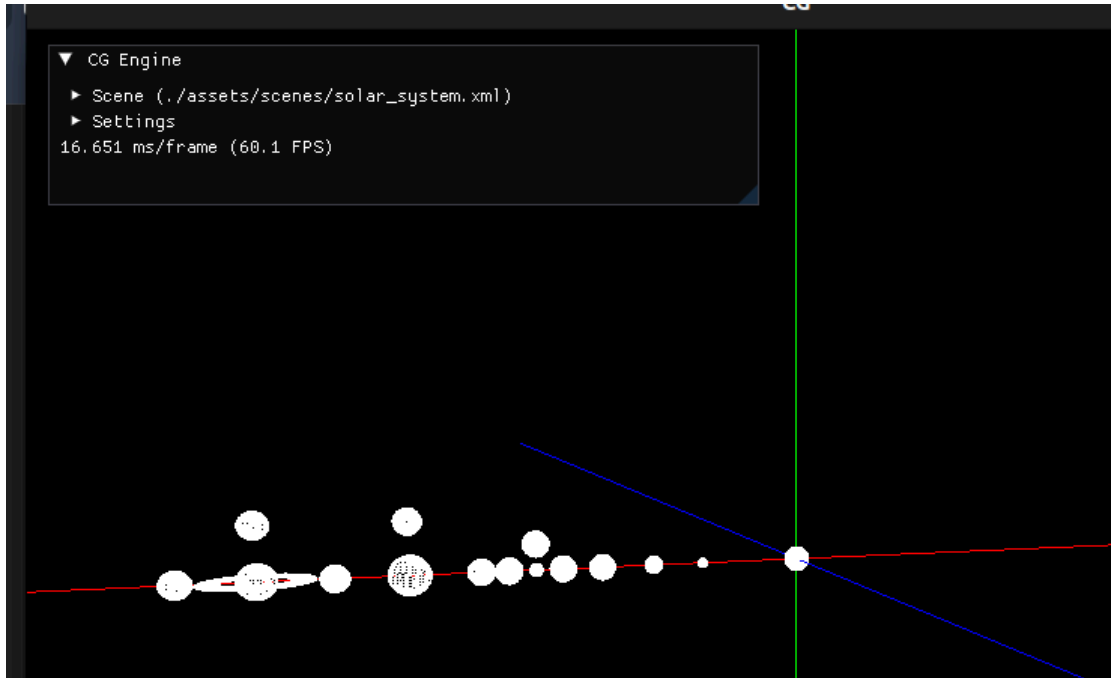


Figura 1: Sistema Solar

3.2. Hierarquia da Cena

Raiz	
├ Sol	sphere (sem transformações)
├ Planeta 1	translate x=8 · scale 0.4
├ Planeta 2	translate x=12 · scale 0.7
├ Planeta 3	translate x=16 · scale 1.0
│ └ Lua	translate x=3 · scale 0.3
├ Planeta 4	translate x=21 · scale 0.5
│ ├ Lua 1	translate x=2 · scale 0.15
│ └ Lua 2	translate y=2 · scale 0.1
├ Planeta 5	translate x=30 · scale 1.5
│ ├ Lua 1	translate x=5 · scale 0.5
│ ├ Lua 2	translate y=4 · scale 0.4
│ └ Lua 3	translate x=-5 · scale 0.4
└ Planeta 6	translate x=40 · scale 1.3
│ └ Anel (torus)	scale x=2 · y=0.2 · z=2
│ └ Lua 1	translate x=5 · scale 0.45
│ └ Lua 2	translate y=4 · scale 0.25

4. Inspetor de Cena

Como funcionalidade extra, o motor inclui um inspetor de cena integrado no painel ImGui, acessível em tempo real durante a execução da aplicação.

4.1. Implementação

O inspetor é implementado pela função `renderImGuiGroupMenu()` em `Engine.cpp`, que recebe um `Group` e percorre recursivamente a sua hierarquia construindo nós de árvore com `ImGui::TreeNode`. O cabeçalho de cada nó indica o número de modelos e de grupos filhos que contém, dando uma visão imediata da complexidade de cada ramo.

Dentro de cada nó são apresentadas três subsecções expansíveis:

- **Transforms** — lista numerada de todas as transformações do grupo, com o tipo e os valores formatados. Por exemplo: *1. Translate (8.00, 0.00, 0.00)* ou *1. Rotate 30.00° em torno de (0.00, 1.00, 0.00)*.
- **Models** — nome do ficheiro de cada modelo carregado no grupo.
- **Child Groups** — grupos filhos, cada um apresentado recursivamente com o mesmo formato.

4.2. Utilidade

Esta funcionalidade é especialmente útil em cenas complexas como o sistema solar, onde a hierarquia tem vários níveis de profundidade. Permite verificar instantaneamente se as transformações foram lidas correctamente do XML, se os modelos foram carregados, e qual a estrutura real da árvore em memória — tudo sem sair da aplicação ou consultar ficheiros externos.

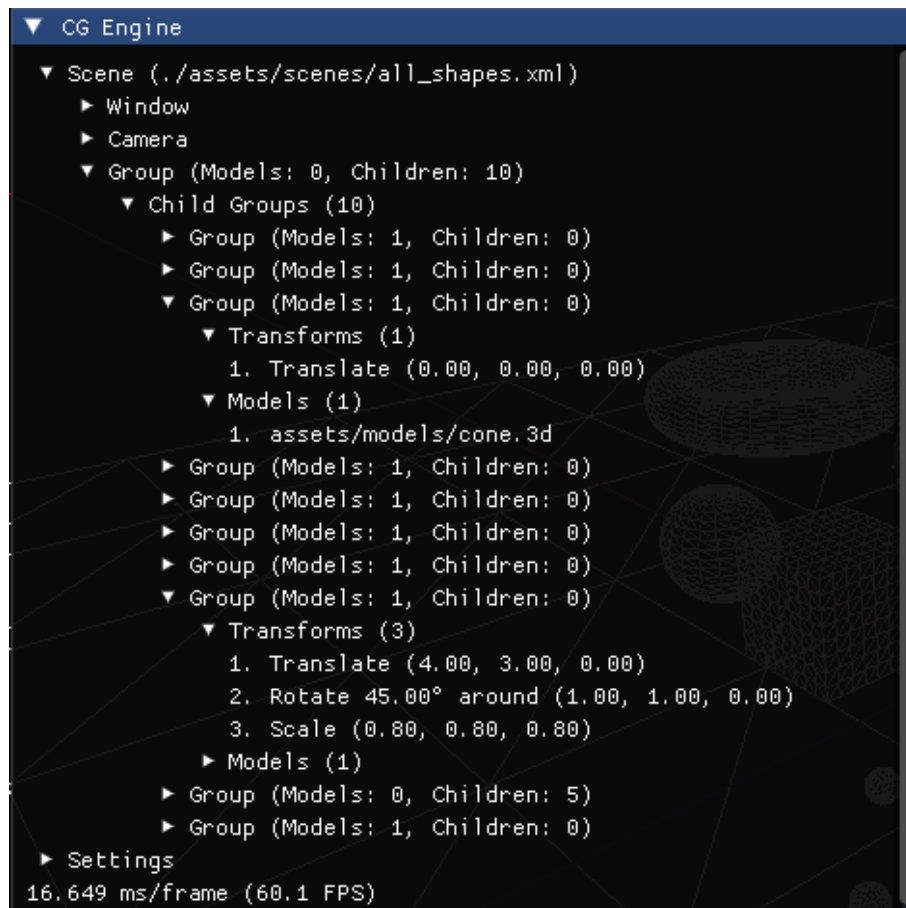


Figura 2: Inspetor de Cena no painel ImGui

5. Testes

De forma a validar as transformações geométricas implementadas, foram realizados quatro testes progressivos, cada um focado num aspeto diferente do sistema.

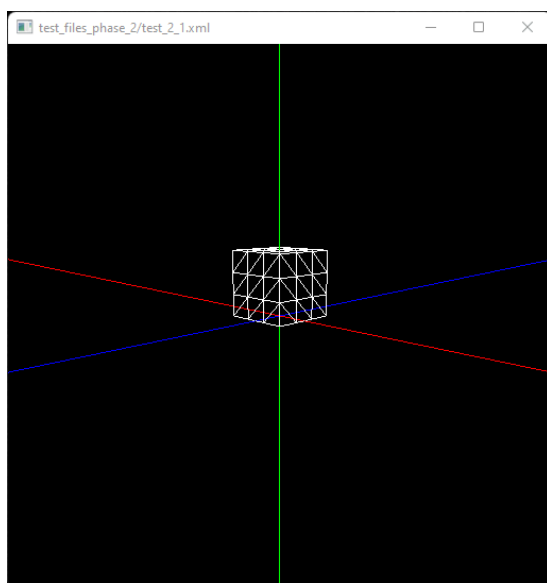


Figura 3: Teste 1 — Cubo com translate, rotate e scale

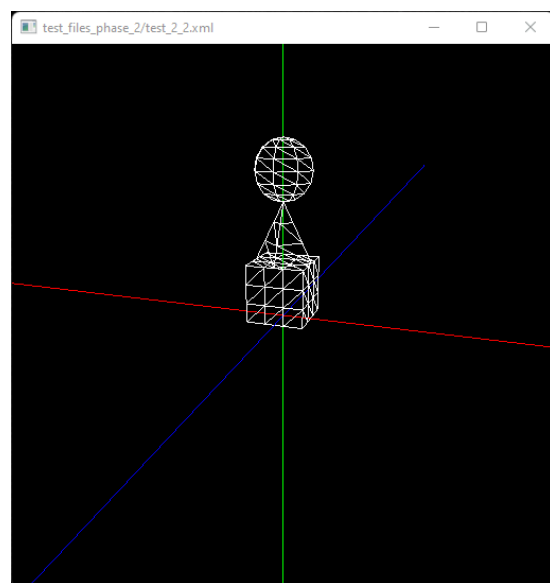


Figura 4: Teste 2 — Hierarquia: caixa, cone e esfera

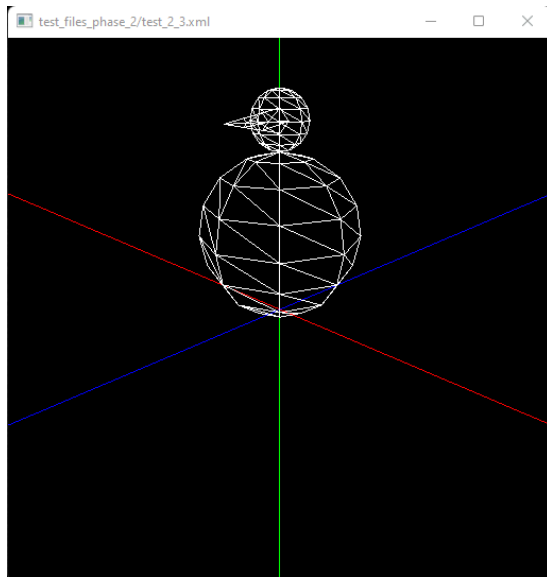


Figura 5: Teste 3 — Boneco de neve

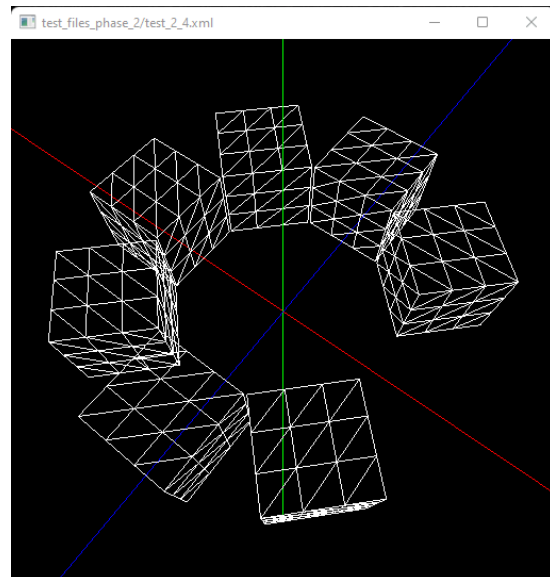


Figura 6: Teste 4 — Caixas em círculo

O **Teste 1** aplica as três transformações em simultâneo a uma caixa — translate, rotate e scale — validando que cada uma funciona individualmente e em combinação.

O **Teste 2** constrói uma hierarquia de três níveis (caixa → cone → esfera), onde cada nível tem a sua própria translação e escala, validando que a herança de transformações entre pai e filho funciona correctamente.

O **Teste 3** compõe um boneco de neve com três esferas de escalas decrescentes e um cone como chapéu filho da cabeça, testando a composição de translate e scale em hierarquia para construir uma figura reconhecível.

O **Teste 4** posiciona seis caixas em arco usando a sequência rotate → translate, demonstrando que a ordem das transformações é relevante e que o motor a respeita — se a ordem fosse invertida, as caixas ficariam todas na mesma linha em vez de em círculo.

6. Conclusão

Esta segunda fase consolidou o motor com suporte a transformações geométricas hierárquicas, permitindo construir cenas complexas de forma estruturada. A leitura recursiva do XML garante que qualquer hierarquia, independentemente da profundidade, é correctamente construída em memória. A cena do sistema solar demonstrou na prática a utilidade da herança de transformações — as luas orbitam os planetas em coordenadas locais, sem necessidade de calcular posições absolutas. O inspetor de cena foi uma ferramenta valiosa durante o desenvolvimento, permitindo analisar a hierarquia em tempo real. A base está preparada para as fases seguintes, onde serão introduzidas animações por curvas de Catmull-Rom, VBOs, iluminação e texturas.